

COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S2
DATE PERFORMED:	August 5, 2026	DATE SUBMITTED:	August 7, 2026
NAME:	Johrel Louie T. Silao	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 5 Queues

1. PROCEDURE OUTPUT

What procedures did you perform?

```

1  #include <iostream>
2  #include <queue>
3
4  void display(std::queue<char> copyQ);
5
6  // |===== IMPLEMENTATION OF sQUEUES =====|
7  int main() {
8
9      // Create an Object:
10     std::queue<char> myQ;
11
12     // Use the enqueue operation
13     myQ.push('I');
14     myQ.push('Y');
15     myQ.push('A');
16
17     // Prints the Front and Back of the queue
18     std::cout << "===== " << std::endl;
19     std::cout << "    QUEUE OPERATIONS    " << std::endl;
20     std::cout << "===== " << std::endl;
21     std::cout << "Current Front is: " << myQ.front() << std::endl;
22     std::cout << "===== " << std::endl;
23     std::cout << "Current Back is: " << myQ.back() << std::endl;
24     std::cout << "===== " << std::endl;
25     display(myQ);
26
27     // Use the dequeue operation
28     myQ.pop();
29     display(myQ);
30     myQ.pop();
31     display(myQ);
32     std::cout << "===== " << std::endl;
33
34     // Check the queue if it is empty
35     std::cout << "Is the queue empty? " << myQ.empty() << std::endl;
36     std::cout << "===== " << std::endl;
37
38
39
40     return 0;
41 }

```

Figure 1.1 Code Snippet: ILO A - Create queue using C++ STL

Observations of Figure 1.1:

- In this code structure, it shows the declaration of 'std::queue<char>' and the code implementation of the STL queue methods, such as 'push()', 'front()', 'back()', 'pop()' and 'empty()'. Successfully demonstrating the proper usage of C++ STL queue methods.

```
=====
                        QUEUE OPERATIONS
=====
Current Front is: I
=====
Current Back is: A
=====
I   Y   A
Y   A
A
=====
Is the queue empty? 0
=====
```

Figure 1.2 Output Snippet: ILO A - Create queue using C++ STL

Observations of Figure 1.2:

- I was able to implement the different types of queue operations such as front, back, push and empty. It works the same as the stack operations but with a little difference.

```

1  #include <iostream>
2  #include "queue1.h"
3
4  int main () {
5      qNode<char>* front = nullptr;
6      qNode<char>* back = nullptr;
7
8      std::cout << "===== " << std::endl;
9      std::cout << "  Testing the Enqueue Operator  " << std::endl;
10     std::cout << "===== " << std::endl;
11
12     enqueue ('I', &front, &back);
13     std::cout << front->data << " " << "back: " << back->data << std::endl;
14     enqueue ('Y', &front, &back);
15     std::cout << front->data << " " << "back: " << back->data << std::endl;
16     enqueue ('A', &front, &back);
17     std::cout << front->data << " " << "back: " << back->data << std::endl;
18
19     std::cout << "===== " << std::endl;
20     std::cout << "  Testing the Dequeue Operator  " << std::endl;
21     std::cout << "===== " << std::endl;
22
23     dequeue(&front, &back);
24     std::cout << front->data << " " << "back: " << back->data << std::endl;
25     dequeue(&front, &back);
26     std::cout << front->data << " " << "back: " << back->data << std::endl;
27     dequeue(&front, &back);
28     dequeue(&front, &back);
29
30     std::cout << "===== " << std::endl;
31     std::cout << "      Display of Elements      " << std::endl;
32     std::cout << "===== " << std::endl;
33
34     std::cout << "Elements: ";
35     displayAll(front);
36
37
38     return 0;
39 }

```

Figure 2.1 Code Snippet: ILO B.1 - Linked List Implementation

Observations of Figure 2.1:

- In this figure, it calls and displays all the header files in the queue linked list, using a template node 'qnode<T>'. The methods enqueue and dequeue are used on how they operate or function.

```

1  #ifndef QUEUELL_H
2  #define QUEUELL_H
3
4  // Create a class node
5  template <typename T>
6  class qNode {
7  public:
8      T data;
9      qNode* next;
10 };
11
12 // New Node Creator
13 template <typename T>
14 qNode<T>* new_node (T newData) {
15     // Allocate a space for the new node
16     qNode<T>* newNode = new qNode<T>;
17
18     // Store the newData to the newNode
19     newNode->data = newData;
20
21     // Point the newNode to the next
22     newNode->next = nullptr;
23
24     return newNode;
25 }
26
27 // Enqueue
28 template <typename T>
29 void enqueue (T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {
30     // Create a New Node
31     qNode<T>* newNode = new_node (newData);
32
33     // Check if the queue is empty
34     if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
35         (*frontPtr) = newNode; // Links the current back to the new node
36         (*backPtr) = newNode; // Assigns the backPtr to the new node
37         return;
38     }
39
40     // If not then insert at the back
41     (*backPtr)->next = newNode;
42     (*backPtr) = newNode;
43
44 }
45
46 // Dequeue
47 template <typename T>
48 void dequeue (qNode<T>** frontPtr, qNode<T>** backPtr) {
49
50     // Check if the queue is empty
51     if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
52         std::cout << "The queue is empty!" << std::endl;
53         return;
54     }
55
56     // Create a temporary variable to store the node to be deleted
57     qNode<T>* deleteTemp = nullptr;
58
59     // Assign the current front to the deleteTemp
60     deleteTemp = (*frontPtr);
61
62     // Check if the current front to the deleteTemp
63     if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr) {
64         // Means only 1 item is inside the queue
65         (*frontPtr) = nullptr;
66         (*backPtr) = nullptr;
67         delete deleteTemp;
68         return;
69     }
70
71     (*frontPtr) = (*frontPtr)->next;
72     deleteTemp->next = nullptr;
73
74     // Delete a node
75     delete deleteTemp;
76 }

```

Figure 2.2 Code Snippet: ILO B.1 - Linked List Implementation

Observations of Figure 2.2:

- In this figure, it shows the manual queue using a template node 'qnode<T>' with front and back pointer while handling the enqueue and dequeue without fixed size limits.

```

78 // Display All
79 template <typename T>
80 void displayAll (qNode<T>* frontPtr) {
81     if (frontPtr == nullptr) {
82         std::cout << "Queue is Empty!" << std::endl;
83         return;
84     }
85     qNode<T>* current = frontPtr;
86     std::cout << "Queue Elements: ";
87     while (current != nullptr) {
88         std::cout << current->data << " ";
89         current = current->next;
90     }
91     std::cout << std::endl;
92 }
93
94 // isEmpty
95 template <typename T>
96 bool isEmpty (qNode<T>* frontPtr) {
97     return (frontPtr == nullptr);
98 }
99
100 #endif

```

Figure 2.3 Code Snippet: ILO B.1 - Linked List Implementation

Observations of Figure 2.3:

- This figure shows the display of all elements in the queue and checks if the queue is empty or not.

```

=====
Testing the Enqueue Operator
=====
I back: I
I back: Y
I back: A
=====
Testing the Dequeue Operator
=====
Y back: A
A back: A
The queue is empty!
=====
Display of Elements
=====
Elements: Queue is Empty!

```

Figure 2.4 Output Snippet: ILO B.1 - Linked List Implementation

Observations of Figure 2.4:

- I was able to implement the different types of queue operations such as front, back, push and empty. It works the same as the stack operations but with a little difference.

```

1  #include <iostream>
2  #include "queueArr.h"
3
4  int main()
5  {
6      // creating an object
7      queueArr<int> q(5);
8
9      // enqueue 10, 20, 30, 40, 50:
10     q.enqueue(1);
11
12     // ===== ARRAY - BASED IMPLEMENTATION =====
13
14     std::cout << "===== " << std::endl;
15     std::cout << " ARRAY - BASED IMPLEMENTATION " << std::endl;
16     std::cout << "===== " << std::endl;
17
18     std::cout << "Front: " << q.Front() << std::endl;
19     std::cout << "Back: " << q.Back() << std::endl;
20     std::cout << "Size: " << q.Size() << std::endl;
21
22     std::cout << "===== " << std::endl;
23
24     q.enqueue(2);
25
26     std::cout << "Front: " << q.Front() << std::endl;
27     std::cout << "Back: " << q.Back() << std::endl;
28     std::cout << "Size: " << q.Size() << std::endl;
29
30     std::cout << "===== " << std::endl;
31
32     q.enqueue(3);
33
34     q.enqueue(4);
35
36     std::cout << "Front: " << q.Front() << std::endl;
37     std::cout << "Back: " << q.Back() << std::endl;
38     std::cout << "Size: " << q.Size() << std::endl;
39
40     std::cout << "===== " << std::endl;
41
42     q.enqueue(5);
43
44     std::cout << "Front: " << q.Front() << std::endl;
45     std::cout << "Back: " << q.Back() << std::endl;
46     std::cout << "Size: " << q.Size() << std::endl;
47
48     std::cout << "===== " << std::endl;
49
50     q.enqueue(6);
51
52     std::cout << "===== ";
53
54     std::cout << "\nremoved: " << q.dequeue() << std::endl;
55     std::cout << "removed: " << q.dequeue() << std::endl;
56
57     std::cout << "===== " << std::endl;
58
59     // testing the clear
60     q.Clear();
61
62     std::cout << "is the queue empty? "
63     | << (q.Empty() ? "yes" : "no")
64     | << std::endl;
65
66     std::cout << "===== " << std::endl;
67
68     return 0;
69 }

```

Figure 3.1 Code Snippet: ILO B.2. Array-Based Implementation

Observations of Figure 3.1:

- This figure demonstrates the 'queueArr<int> q(5)' object, a fixed sized integer queue that only has a maximum of 5 elements. Following is the execution of increment of elements from 1 to 5, intentionally adding a 6th element to verify that the overflow protection is functioning properly. Also, the 'front()', 'back()', 'size()', 'dequeue()', 'clear()', and 'empty()' are executed.

```

1  #ifndef QUEUEARR_H
2  #define QUEUEARR_H
3  #include <iostream>
4
5  // Array-Based Circular Queue
6  template <typename T>
7  class queueArr {
8  private:
9      // POINTER TO DYNAMICALLY ALLOCATE ARRAY
10     T* q_arr;
11
12     // MAXIMUM NUMBER OF ELEMENTS A QUEUE CAN HOLD
13     size_t q_capacity;
14
15     // CURRENT NUMBER OF ELEMENTS IN THE QUEUE
16     size_t q_size;
17
18     // INDEX OF THE FRONT ELEMENT IN THE QUEUE
19     int q_front;
20
21     // INDEX OF THE BACK ELEMENT IN THE QUEUE
22     int q_back;
23
24 public:
25
26     // CONSTRUCTOR
27     queueArr(size_t capacity) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
28         // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
29         q_arr = new T[q_capacity];
30     }
31
32     // COPY CONSTRUCTOR
33     queueArr(const queueArr& other) : q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_back(other.q_back) {
34         // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
35         q_arr = new T[q_capacity];
36         // COPY ELEMENTS FROM OTHER QUEUE
37         for (size_t i = 0; i < q_size; ++i) {
38             q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
39         }
40     }
41
42     // COPY ASSIGNMENT OPERATOR
43     queueArr& operator=(const queueArr& other) {
44         if (this != &other) {
45             // DEALLOCATE CURRENT ARRAY
46             delete[] q_arr;
47
48             // COPY CAPACITY AND SIZE
49             q_capacity = other.q_capacity;
50             q_size = other.q_size;
51             q_front = other.q_front;
52             q_back = other.q_back;
53
54             // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
55             q_arr = new T[q_capacity];
56             // COPY ELEMENTS FROM OTHER QUEUE
57             for (size_t i = 0; i < q_size; ++i) {
58                 q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
59             }
60         }
61         return *this;
62     }

```

Figure 3.2 Code Snippet: ILO B.2. Array-Based Implementation

Observations of Figure 3.2:

- This figure defines the template 'queueArr<T>' that will allow array-based queues.

```

64 // DESTRUCTOR
65 ~queueArr() {
66     // DEALLOCATE MEMORY FOR THE QUEUE ARRAY
67     delete[] q_arr;
68 }
69
70 // QUEUE OPERATIONS
71 bool Empty();
72 bool Full();
73 size_t Size();
74 void Clear();
75 T Front();
76 T Back();
77 void enqueue(T value);
78 T dequeue();
79 };
80
81 // Empty()
82 template <typename T>
83 bool queueArr<T>::Empty() {
84     // CHECK IF EMPTY
85     return q_size == 0;
86 }
87
88 // Full()
89 template <typename T>
90 bool queueArr<T>::Full() {
91     // CHECK IF REACHES MAX CAPACITY
92     return q_size == q_capacity;
93 }
94
95 // Size()
96 template <typename T>
97 size_t queueArr<T>::Size() {
98     // RETURN CURRENT SIZE OF QUEUE
99     return q_size;
100 }
101
102 // Front()
103 template <typename T>
104 T queueArr<T>::Front() {
105     // Check if the queue is empty
106     if (Empty()) {
107         std::cout << "Queue is Empty\n";
108         return T(); // Return default value of type T
109     }
110     // Return the front
111     return q_arr[q_front];
112 }
113
114 // Back()
115 template <typename T>
116 T queueArr<T>::Back() {
117     // Check if the queue is empty
118     if (Empty()) {
119         std::cout << "Queue is Empty\n";
120         return T(); // Return default value of type T
121     }
122     // Return the back
123     return q_arr[q_back];
124 }

```

Figure 3.3 Code Snippet: ILO B.2. Array-Based Implementation

Observations of Figure 3.3:

- This implements the class 'destructor()' to release dynamically allocated heap array memory and prevent any memory leaks. Additionally, the element inspection methods provide elements access through the front and back that will return.

```

126 // Clear()
127 template <typename T>
128 void queueArr<T>::Clear() {
129     // Reset the queue to its initial state
130     q_size = 0;
131     q_front = 0;
132     q_back = -1; // Set back to -1 since there are no elements
133 }
134
135 // Enqueue()
136 template <typename T>
137 void queueArr<T>::enqueue(T value) {
138     // Check if the queue is full
139     if (Full()) {
140         std::cout << "Queue is Full\n";
141         return;
142     }
143     // Increment the back
144     q_back = (q_back + 1) % q_capacity;
145     // Add the new value to the back of the queue
146     q_arr[q_back] = value;
147     // Increment the size of the queue
148     q_size++;
149 }
150
151 // Dequeue()
152 template <typename T>
153 T queueArr<T>::dequeue() {
154     // Check if the queue is empty
155     if (Empty()) {
156         std::cout << "Queue is Empty\n";
157         return T(); // Return default value of type T
158     }
159     // Store the front value
160     T value = q_arr[q_front];
161     // Increment the front index in a circular manner
162     q_front = (q_front + 1) % q_capacity;
163     // Decrement the size of the queue
164     q_size--;
165     // Reset the indexes
166     if (Empty()) Clear();
167     // Return the dequeued value
168     return value;
169 }
170
171 #endif // QUEUEARR_H

```

Figure 3.4 Code Snippet: ILO B.2. Array-Based Implementation

Observations of Figure 3.4:

- The 'clear()' will reset the parameters to their default empty state without deallocating the memory.

```
=====
  ARRAY – BASED IMPLEMENTATION
=====
Front: 1
Back: 1
Size: 1
=====
Front: 1
Back: 2
Size: 2
=====
Front: 1
Back: 4
Size: 4
=====
Front: 1
Back: 5
Size: 5
=====
Queue is Full
=====
removed: 1
removed: 2
=====
is the queue empty? yes
=====
```

Figure 3.5 Output Snippet: ILO B.2. Array-Based Implementation

Observations of Figure 3.5:

- This output figure shows that the code implementation was executed successfully without any errors.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues

Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.

You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

Output Analysis:

- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

```

1  #include <iostream>
2  #include "printer.h"
3
4  // Simulating a Scenario
5  int main() {
6      Printer officePrinter;
7
8      std::cout << "===== " << std::endl;
9      std::cout << "      OFFICE PRINTER SIMULATION      " << std::endl;
10     std::cout << "===== " << std::endl;
11
12     // Adding print requests from users
13     officePrinter.addJob(1, "Johrel (CPE)", 5);
14     officePrinter.addJob(2, "Louie (IT)", 12);
15     officePrinter.addJob(3, "Silao (CS)", 2);
16
17     // Printing individual job
18     std::cout << "\n--- Processing ---\n";
19     officePrinter.processNextJob();
20
21     // Adding more requests while queue is active
22     std::cout << "\n--- Incoming Requests ---\n";
23     officePrinter.addJob(4, "Johlo (ARCHITECT)", 8);
24     officePrinter.addJob(5, "Joh (ENGINEER)", 15);
25
26     // Printing all remaining jobs
27     officePrinter.processAllJobs();
28
29     return 0;
30 }

```

Figure 4: Code Snippet: ILO C - main.cpp

Observations of Figure 4:

- This figure is setting up the scenario that calls the printer header, queues the print tasks from different users, processes their jobs and accepts the requested print tasks and clears the remaining pending jobs.

```
1  #ifndef JOB_H
2  #define JOB_H
3
4  #include <iostream>
5  #include <string>
6
7  // 1. Creating a class called Job
8  class Job {
9  private:
10     int jobID;
11     std::string userName;
12     int numberofpage;
13
14 public:
15     Job(int id = 0, const std::string& user = "", int pages = 0)
16         : jobID(id), userName(user), numberofpage(pages) {}
17
18     void printDetails() const {
19         std::cout << "Job ID: " << jobID << " | User: " << userName << " | Pages: " << numberofpage;
20     }
21
22     int getPageCount() const { return numberofpage; }
23     std::string getUser() const { return userName; }
24     int getJobID() const { return jobID; }
25 };
26
27 #endif
```

Figure 5: Code Snippet: ILO C - job.h

Observations of Figure 5:

- Processes the job header file information, stores the print data including the jobID, username, and the number of pages with their helper functions.

```

1  #ifndef PRINTER_H
2  #define PRINTER_H
3
4  #include <iostream>
5  #include "job.h"
6
7  // Node structure for custom Linked List Queue
8  struct JobNode {
9      Job job;
10     JobNode* next;
11     JobNode(const Job& j) : job(j), next(nullptr) {}
12 };
13
14 // 2. Creation of Printer Class and Implementation
15 class Printer {
16 private:
17     JobNode* frontPtr;
18     JobNode* backPtr;
19     int pendingJobsCount;
20
21 public:
22     Printer() : frontPtr(nullptr), backPtr(nullptr), pendingJobsCount(0) {}
23
24     ~Printer() {
25         while (hasPendingJobs()) {
26             processNextJob();
27         }
28     }
29
30     bool hasPendingJobs() const {
31         return frontPtr != nullptr;
32     }
33
34     // Enqueue
35     void addJob(int id, const std::string& user, int pages) {
36         Job newJob(id, user, pages);
37         JobNode* newNode = new JobNode(newJob);
38
39         if (hasPendingJobs()) {
40             backPtr->next = newNode;
41             backPtr = newNode;
42         } else {
43             frontPtr = newNode;
44             backPtr = newNode;
45         }
46         pendingJobsCount++;
47         newJob.printDetails();
48         std::cout << " added to print queue." << std::endl;
49     }
50
51     // Dequeue
52     void processNextJob() {
53         if (!hasPendingJobs()) {
54             std::cout << "No pending print jobs." << std::endl;
55             return;
56         }
57
58         JobNode* temp = frontPtr;
59         std::cout << "Printing job for " << temp->job.getUser() << " (" << temp->job.getPageCount() << " pages)... Done!\n";
60
61         frontPtr = frontPtr->next;
62         if (frontPtr == nullptr) {
63             backPtr = nullptr;
64         }
65
66         delete temp;
67         pendingJobsCount--;
68     }
69
70     void processAllJobs() {
71         std::cout << "\n--- Print Job Execution ---\n";
72         while (hasPendingJobs()) {
73             processNextJob();
74         }
75         std::cout << std::endl;
76         std::cout << "--- All Printed Jobs Are Completed ---";
77     }
78 };
79
80 #endif // PRINTER_H

```

Figure 6: Code Snippet: ILO C - printer.h

Observations of Figure 6:

- This declares the linked list structure and class logic.

```
=====
OFFICE PRINTER SIMULATION
=====
Job ID: 1 | User: Johrel (CPE) | Pages: 5 added to print queue.
Job ID: 2 | User: Louie (IT) | Pages: 12 added to print queue.
Job ID: 3 | User: Silao (CS) | Pages: 2 added to print queue.

--- Processing ---
Printing job for Johrel (CPE) (5 pages)... Done!

--- Incoming Requests ---
Job ID: 4 | User: Johlo (ARCHITECT) | Pages: 8 added to print queue.
Job ID: 5 | User: Joh (ENGINEER) | Pages: 15 added to print queue.

--- Print Job Execution ---
Printing job for Louie (IT) (12 pages)... Done!
Printing job for Silao (CS) (2 pages)... Done!
Printing job for Johlo (ARCHITECT) (8 pages)... Done!
Printing job for Joh (ENGINEER) (15 pages)... Done!

--- All Printed Jobs Are Completed ---%
```

Figure 7: Output Snippet: ILO C

Observations of Figure 7:

- This output shows the complete execution of the program and successfully displays the expected output of the program.

Conclusion:

- This hands-on activity demonstrates how to properly and creatively use queues using C++ STL. I also was able to apply the First In, First Out (FIFO) that ensures the data is accessed and removed in an order when it was inserted. Additionally, I was able to use arrays and linked lists to create a stack. The printer simulation demonstrates how queues are important where shared documents or data must be fairly managed with incoming requests in chronological order. All in all, I finished the activity and encountered multiple errors in the process.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.